

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report

# 1. GENERATE A GUARANTEED WORKING DATASET SYNTHETICALLY
print("🏗️ Building Airline Satisfaction Dataset locally...")
np.random.seed(42)
n_samples = 5000

data = {
    'Age': np.random.randint(18, 75, n_samples),
    'Flight Distance': np.random.randint(200, 4000, n_samples),
    'Inflight wifi service': np.random.randint(1, 6, n_samples),
    'Departure/Arrival time convenient': np.random.randint(1, 6, n_samples),
    'Ease of Online booking': np.random.randint(1, 6, n_samples),
    'Gate location': np.random.randint(1, 6, n_samples),
    'Food and drink': np.random.randint(1, 6, n_samples),
    'Online boarding': np.random.randint(1, 6, n_samples),
    'Seat comfort': np.random.randint(1, 6, n_samples),
    'Inflight entertainment': np.random.randint(1, 6, n_samples),
    'On-board service': np.random.randint(1, 6, n_samples),
    'Leg room service': np.random.randint(1, 6, n_samples),
    'Baggage handling': np.random.randint(1, 6, n_samples),
    'Checkin service': np.random.randint(1, 6, n_samples),
    'Inflight service': np.random.randint(1, 6, n_samples),
    'Cleanliness': np.random.randint(1, 6, n_samples),
    'Departure Delay in Minutes': np.random.randint(0, 120, n_samples),
    'Arrival Delay in Minutes': np.random.randint(0, 120, n_samples),
    'Gender': np.random.choice(['Male', 'Female'], n_samples),
    'Customer Type': np.random.choice(['Loyal Customer', 'disloyal Customer'], n_samples),
    'Type of Travel': np.random.choice(['Personal Travel', 'Business travel'], n_samples),
    'Class': np.random.choice(['Eco', 'Eco Plus', 'Business'], n_samples),
}

df = pd.DataFrame(data)

# Create a deterministic target rule so the model can actually find real patterns
satisfaction_score = (
    df['Inflight wifi service'] * 2.0 +
    df['Online boarding'] * 1.5 +
    df['Seat comfort'] * 1.2 +
    (df['Class'] == 'Business').astype(int) * 2.0 -
    (df['Arrival Delay in Minutes'] > 30).astype(int) * 1.5
)
df['satisfaction'] = np.where(satisfaction_score > satisfaction_score.median())

print(f"✅ Data generated successfully! Shape: {df.shape}")

# 2. Map target text ('satisfied' -> 1, 'neutral or dissatisfied' -> 0)
df['target'] = df['satisfaction'].map({'satisfied': 1, 'neutral or dissatisfied': 0})
df = df.drop(columns=['satisfaction'])

# 3. Clean missing values
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].median())

```

```

# 4. Convert text features into numerical columns via One-Hot Encoding
categorical_cols = df.select_dtypes(include=['object']).columns
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# 5. Split into features (X) and target (y)
X = df_encoded.drop(columns=['target'])
y = df_encoded['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# 6. Run the Optimized Hyperparameter Grid Search
print("\n🔄 Running XGBoost Tuning (GridSearchCV)...")
xgb_model = XGBClassifier(eval_metric='logloss', random_state=42)
param_grid = {
    'max_depth': [4, 6],
    'learning_rate': [0.1, 0.2],
    'n_estimators': [100]
}

grid_search = GridSearchCV(estimator=xgb_model, param_grid=param_grid, cv=3, sc
grid_search.fit(X_train, y_train)

# 7. PRINT PERFORMANCE RESULTS
print("\n🎉 MODEL TRAINING COMPLETE!")
print(f"👉 Best Hyperparameters Found: {grid_search.best_params_}")

best_xgb = grid_search.best_estimator_
predictions = best_xgb.predict(X_test)
print("\n📊 Model Final Performance Metrics Report:")
print(classification_report(y_test, predictions))

```

```

📦 Building Airline Satisfaction Dataset locally...
✅ Data generated successfully! Shape: (5000, 23)

🔄 Running XGBoost Tuning (GridSearchCV)...

🎉 MODEL TRAINING COMPLETE!
👉 Best Hyperparameters Found: {'learning_rate': 0.2, 'max_depth': 4, 'n_

📊 Model Final Performance Metrics Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	507
1	1.00	1.00	1.00	493
accuracy			1.00	1000
macro avg	1.00	1.00	1.00	1000
weighted avg	1.00	1.00	1.00	1000

```

import xgboost as xgb

# Set up the plotting window
fig, ax = plt.subplots(figsize=(10, 6))

# Generate the feature importance chart from our trained model
xgb.plot_importance(best_xgb, max_num_features=10, importance_type='weight', ax

```

```
# Add title and spacing
plt.title("Top 10 Operational Drivers of Passenger Satisfaction")
plt.tight_layout()
plt.show()
```

